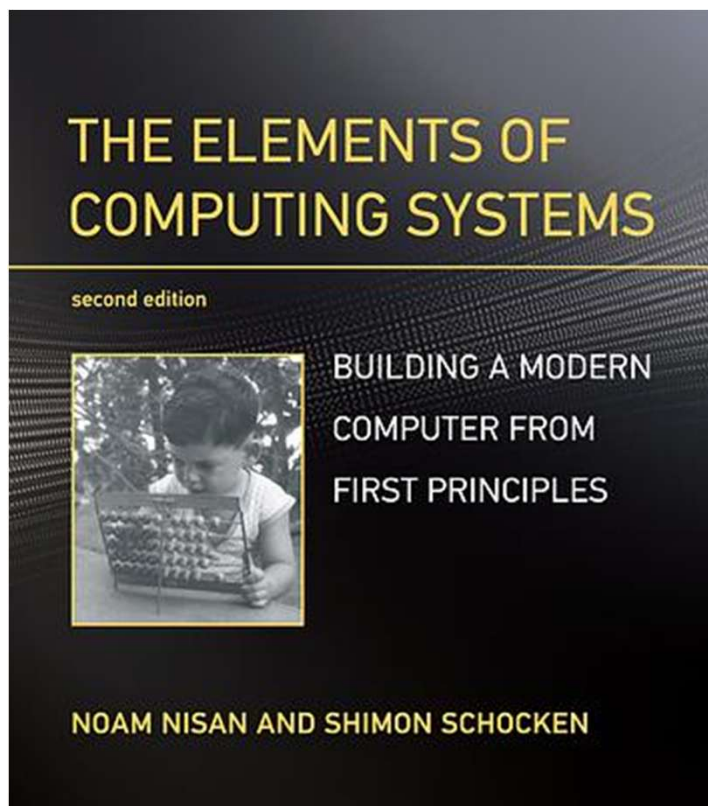


프로젝트6

Nand2Tetris _ Overview



개요

프로젝트명	Nand2Tetris
개발 기간	2024.10 ~ 2025.2
Tool	Visual Studio, C#
관련 기관	개인 프로젝트
담당 역할	소자 개발, ALU 개발, 어셈블러 개발

소개

이 프로젝트는 “The Elements of Computing Systems – Noam Nisan “ 책에서 설정한 컴퓨터 아키텍처를 바탕으로 Nand 소자부터 어셈블러까지 개발한 프로젝트이다. 책에서는 Hack 컴퓨터의 아키텍처와 Hack 어셈블리의 문법을 제공한다. 이를 참고하여 Hack 어셈블러를 개발하였다.

개발 순서

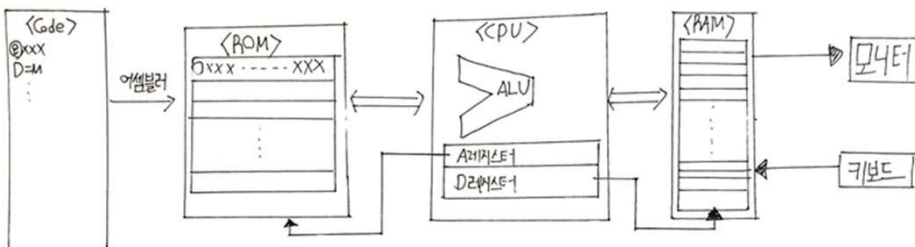
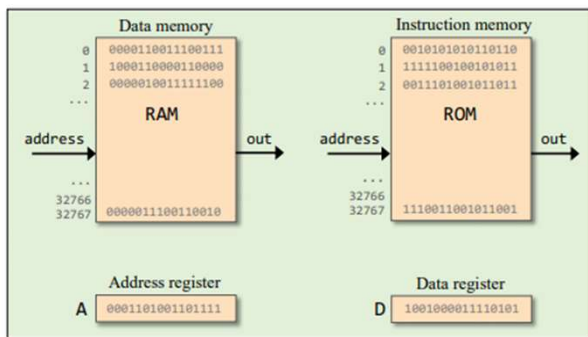
- HDL로 논리 게이트 개발
- 논리 게이트로 가산기, 증분기 개발
- ALU 개발
- 레지스터, RAM 개발
- 컴퓨터 개발(일명 ‘Hack 컴퓨터’)
- Hack 어셈블러 개발

프로젝트6

HACK Computer

컴퓨터 구조

Hack 컴퓨터는 명령어 메모리 모듈(16비트, ROM 역할)과 데이터 메모리 모듈(16비트, RAM 역할)을 가지고 있는 폰 노이만 구조이다. 메모리에 접근하는 Address는 15비트이다.



Hack 어셈블리

이 컴퓨터의 기계어에는 M명령어, A명령어, C명령어, 라벨
4가지 타입이 있다.

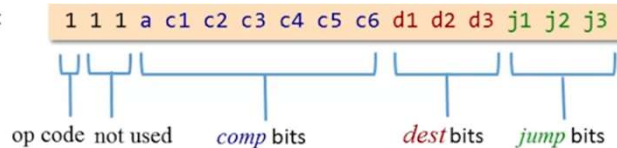
M명령어는 메모리를 M으로 표기하는 것을 말한다.

A 명령어는 @상수, @심볼 2가지를 사용한다.

C 명령어는 연산을 수행하는 명령어로
(저장장소/연산식/점프식)의 순서를 가지는
111a_cccc_ccdd_djjj인 기계어로 변환된다.

라벨은 반복문을 위한 프로그램 회귀 위치이다.

Binary syntax:



```
// Computes:  $RAM[2] = RAM[0] + RAM[1] + 17$ 
// D =  $RAM[0]$ 
@0
D=M
// D = D +  $RAM[1]$ 
@1
D=D+M
// D = D + 17
@17
D=D+A
//  $RAM[2] = D$ 
@2
M=D
```

프로젝트6

Nand2Tetris _ HACK Assembler

HACK Assembler

어셈블러 구조

앞선 Hack 기계어와 Hack 컴퓨터의 제어코드를 고려하여 어셈블러를 개발한다.

Hack Assembler의 기능은 3가지 소스코드로 나눈다.

- Parser : 명령어를 읽고 파싱
- Code : 이진 코드 작성
- SymbolTable : symbol을 다룬다.

Hack Assembler의 동작은 Initialize, First Pass, Second Pass 순서로 이루어진다.

- Initialize : 기계어를 읽고 일부 심볼을 테이블에 저장.
- First Pass : 프로그램 줄을 하나씩 읽고 라벨 선언만 집중해서 파악한다.
- Second Pass : 명령어를 읽고 파싱. 명령어를 이진코드로 변환한다.

시행 착오

가장 중요하면서도 어려웠던 부분은 프로그램을 구현하기 이전에 Hack 기계어와 Hack 컴퓨터의 제어코드를 정확히 이해하는 과정이었다.

어셈블러는 단순한 텍스트 변환 프로그램처럼 보이지만, 실제로는 CPU가 명령어를 어떻게 해석하고 실행하는지에 대한 이해가 선행되어야만 구현이 가능했다.

이를 위해 Hack CPU의 회로 구조를 다시 살펴보며 각 명령어와 제어 비트의 의미를 정리했고, 이해한 내용을 엑셀로 체계적으로 정리하는 과정을 반복하였다.

또한 시간이 지나더라도 구조를 쉽게 다시 이해할 수 있도록 코드에 충분한 주석을 달고 관련 내용을 PDF 문서로 정리해 두었다. 이러한 준비 과정 덕분에 이후 어셈블러 개발을 비교적 수월하게 진행할 수 있었다.

Hack Assembly는 제한된 명령어 집합을 기반으로 작성되기 때문에 각 명령어가 실제 프로그램 흐름 속에서 어떤 역할을 수행하는지 지속적으로 확인해야 했다. RAM 구조를 직접 그려가며 프로그램의 실행 흐름을 추적하는 방식으로 코드의 논리를 이해하였다.

또한 이진 코드를 직접 다루는 과정에서도 어려움이 있었다. Hack CPU에서 사용하는 명령어와 이에 대응하는 이진 코드가 미리 정의되어 있었기 때문에, 이를 디서너리 형태로 저장하여 어셈블러를 구현하였다. 그러나 출력되는 결과가 모두 이진 코드이다 보니 오류가 발생했을 때 원인을 파악하기가 쉽지 않았다. 실제로 디서너리에 저장된 매핑 값 자체가 잘못되어 있는 경우가 많았고, 이 부분을 검증하고 수정하는 과정에 가장 많은 시간을 투자하게 되었다.

구현한 Assembler를 이용하여 Hack Assembly 프로그램을 입력 받아 Hack CPU에서 실행 가능한 16-bit 기계어(binary machine code)로 변환하는 Assembler를 구현하였다.